InRole: A Recruitment Service for Employers and Applicants

Design Report

Team: Clovers

Members: Mahad Maqsood (23393586), Jennifer Joby (23323553), Hamza Yasin (23481144), Sheazy Rato (23346006)

1 Project Idea

Situation and problem. Employers advertise vacancies and often receive many applications before the closing date. Applications for one vacancy move at different speeds, applicants apply for several roles at once, and several members of an employer's staff may review candidates. Managed through email and spreadsheets, this causes three problems. Applicants cannot see what happened to an application. Staff decide and write to applicants inconsistently, because nothing records who changed an application or why. Expired, misleading or inappropriate vacancies and employer accounts stay visible, because there is no defined way to report and remove them. InRole gives every application one shared record with a visible history, and gives each user only the actions their responsibility needs.

User roles.

- **Applicant.** Finds open vacancies, applies with a CV and follows each application. Sees only their own applications and the notes staff mark as visible to applicants.
- **Recruiter (employer staff).** Publishes and closes the employer's vacancies and moves applications through the stages. The only role that changes a status, so one accountable person makes each decision.
- **Reviewer (employer staff).** Reads the employer's applicants and adds internal notes and a recommendation. Cannot change a status or publish a vacancy.
- **Administrator.** Verifies employers, handles reports, hides or removes vacancies and suspends employers. Takes no part in hiring decisions.

Usefulness and coverage. InRole covers the path from publishing a vacancy to a recorded decision, plus the moderation needed to keep listings trustworthy. Applicants see where each application stands and employers keep decisions consistent. Payments and video interviews are out of scope. Interview booking, messaging, reviewer assignment, CV parsing and candidate matching are possible extensions.

2 Main Functions

Table 1 lists the main functions. *Essential* functions are those without which an applicant cannot find a vacancy, apply and learn the outcome, or the service cannot deal with an unsafe vacancy. *Important* functions improve consistency but the service works without them. *Optional* functions are extensions.

Function	Who	What they can do and the result	Priority	Rules
F1–F2 Accounts and access	All users	Register and log in. Each role sees only its own pages. Employer accounts start as pending.	Essential	R1, R6
F3 Profile and CV	Applicant	Keep details and upload a CV (type and size checked) for use when applying.	Essential	R8
F4 Manage vacancies	Recruiter	Create, edit and close vacancies (draft, open, closed) for their own employer.	Essential	R1, R2, R6
F5–F6 Search and view	Applicant, visitor	Search open vacancies by keyword, location and type, and see the closing date and state.	Essential	R2, R6
F7 Apply	Applicant	Submit one application with a CV and cover note. It starts as Submitted.	Essential	R2, R3, R8
F8, F14 Dashboard, timeline, withdraw	Applicant	See each application's stage and history. Withdraw an active one (F14, important).	Essential	R1, R4, R5
F9 Applicant list	Recruiter, reviewer	Filter, sort and page through applicants, with counts per stage.	Essential	R1
F10 Update status	Recruiter	Move an application to the next allowed stage with a note. The timeline gains an entry.	Essential	R4, R5
F15 Reviewer feedback	Reviewer	Add internal notes and a recommendation, hidden from applicants.	Essential	R1, R4
F11–F13 Report and moderate	Any user, administrator, system	Report a vacancy or employer. The administrator hides or removes vacancies, verifies or suspends employers and records the decision. A removal closes active applications with an explanation.	Essential	R5, R6
F16, F21 Templates, email	Recruiter, system	Standard decision wording, and an email on status change (new technology, Section 4).	Important	R5
F17–F20 Extensions	Various	Interview booking, messages, reviewer assignment, saved vacancies.	Optional	R7

Table 1: Main functions, their users, results, priorities and rules.

Priorities. The timeline (F8) and recorded status changes (F10) are essential because they solve the central problems of applicants not knowing and staff acting inconsistently. Reviewer feedback (F15) is essential because several staff are involved, and moderation (F11–F13) because the service must respond to misleading vacancies and accounts. Interview booking is an extension because the core problems do not depend on it.

Rules.

- **R1 Ownership and visibility.** Staff see only their own employer's vacancies and applicants. Applicants see only their own applications. Internal notes are never shown to applicants.
- **R2 Closing date.** No application is accepted after the closing date, and closed or expired vacancies are not listed as open. Expiry is checked from the closing date on every request.
- **R3 One application** per applicant per vacancy, but an applicant may apply to many vacancies.
- **R4 Status order and authority.** Stages are Submitted, Under review, Shortlisted, Interview and Offer. The recruiter may reject at any active stage and the applicant may withdraw. Only recruiters change status.
- **R5 Recorded history.** Every status change stores who made it, when, and a note.
- **R6 Approval and moderation.** An employer must be verified before its vacancies are public. A reported vacancy may be hidden until the administrator decides, and an employer may be suspended.
- **R7 Availability (extension only).** An interview slot can be booked by one applicant.
- **R8 Private CVs.** Only the applicant and staff of the vacancy's employer can open a CV, which is kept outside publicly served files.

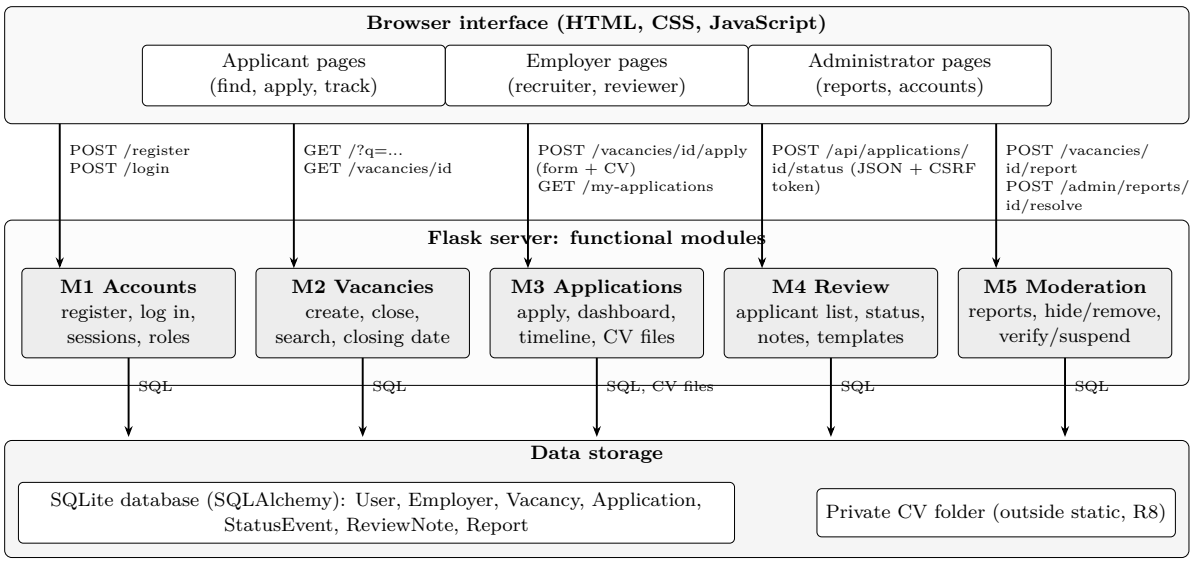


Figure 1: Main parts of InRole and the requests and data they exchange. Arrow labels show the main requests from the browser to each module.

3 Application Design

Application structure. Figure 1 shows the parts and their connections. The browser interface has pages for each role. HTML and CSS present them, and JavaScript filters the vacancy list, checks forms before sending and sends status updates with Fetch. Every request reaches one Flask server whose five modules each own part of the process: **M1 Accounts** (registration, login, sessions and roles; applies R1), **M2 Vacancies** (create, close and search; evaluates R2), **M3 Applications** (apply, dashboard and timeline; serves CVs under R8), **M4 Review** (applicant lists, status changes and notes; enforces R4 and R5) and **M5 Moderation** (reports, hiding and removal, verification; applies R6). All modules read and write one SQLite database through SQLAlchemy, and M3 also writes CVs to a private folder.

Data. Table 2 lists the stored records and the functions that use or update them.

Record	Information stored	Functions that use or update it
User	Name, email, password hash, role, employer and staff level (staff only).	F1–F2 create and check accounts. All modules read the role (R1).
Employer	Name, description, status (pending, verified, suspended).	F1 creates it as pending. F11–F12 verify or suspend. F4 requires verified.
Vacancy	Employer, creator, title, description, location, type, closing date, status (draft, open, closed, hidden, removed).	F4 creates, edits, closes. F5–F6 show. F7 checks open and date. F12–F13 hide or remove.
Application	Applicant, vacancy, CV filename, cover note, current status, time. One per applicant and vacancy.	F7 creates. F8 shows. F9 lists. F10 updates status. F14 withdraws. F13 closes.
StatusEvent	Application, old and new status, who changed it (user or system), applicant note, internal note, time.	F10, F13, F14 add. F8 shows as the timeline. F9 shows to staff.
ReviewNote	Application, reviewer, text, recommendation, time.	F15 adds. Shown to staff only (R1).
Report	Reporter, vacancy or employer, reason, status, resolved by, note, time.	F11 creates. F12 resolves. A removal triggers F13.

Table 2: Records stored by the application.

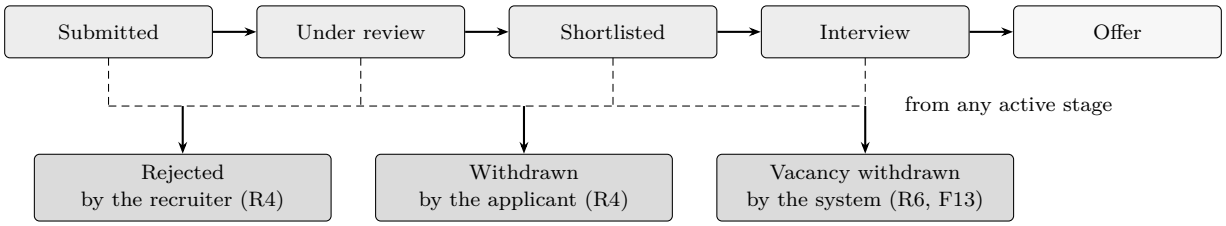


Figure 2: Application status stages. Dashed lines show changes that can happen from any active stage.

The records are linked by identifiers. An employer has many staff and vacancies, a vacancy has many applications, an applicant has many applications, and each application has many status events and review notes. A report points to one vacancy or employer. The applicant's timeline is built from the same StatusEvent records that staff add, so both sides see one consistent history.

User workflow. An applicant applies and is taken to a decision.

1. The applicant searches (F5, M2) and opens a vacancy (F6).
2. The applicant submits the form with a CV (F7). M3 checks the login and role, that the vacancy is open and before its closing date (R2), that no application exists yet (R3), and that the fields and file are valid.
3. M3 creates an Application (Submitted) and a first StatusEvent, shown on the dashboard (F8).
4. A recruiter opens the applicant list (F9). M4 checks that the vacancy belongs to the recruiter's employer (R1). Reviewers add internal notes (F15).
5. The recruiter moves the application to the next stage with a note. The browser sends JSON with a CSRF token. M4 checks the role and the allowed order (R4), updates the Application and adds a StatusEvent (R5).
6. Steps 4 and 5 repeat through the stages in Figure 2 until an Offer or a rejection. The applicant's timeline shows each step.

The applicant sees the decision and its history, and the employer holds one recorded decision with its author.

Exception: a reported vacancy is removed while applications are in progress. An applicant reports the vacancy (F11). M5 stores a Report and, under R6, may hide the vacancy until it is reviewed. The administrator examines the report in the moderation queue (F12) and removes the vacancy. M5 blocks new applications and, for every active application, sets the status to Vacancy withdrawn with a system StatusEvent that explains why (F13, Figure 2). Applicants see this on their timeline and staff can no longer change those applications. If the report is dismissed, the vacancy is restored and nothing changes.

4 Technology Choices

Browser. HTML structures the pages, CSS gives a responsive layout, and JavaScript filters vacancies as the user types, checks forms and uses Fetch with JSON to update statuses and load timelines without a reload.

Server. Python with Flask provides modules M1–M5. Jinja builds the pages, Flask-WTF validates forms and adds CSRF tokens, Flask-Login manages sessions and restricts routes, and Werkzeug hashes passwords.

Data storage. SQLite, through Flask-SQLAlchemy, stores the records in Table 2 with foreign key, uniqueness and CHECK constraints. CVs are kept outside the public static files and sent only through a route that checks R8.

Why these suit the application. The data are relational, every route needs a permission check, the whole application runs on one machine for demonstration, and all of these technologies were taught in class.

Technology not covered in class. We propose the following.

- **Flask-Migrate.** Purpose: versioned database changes. Benefit: several developers will keep changing the tables, and `db.create_all()` does not alter existing tables, so otherwise each change would delete the test data. Learning: one package and three commands. Fallback: a reset-and-seed script.
- **pytest.** Purpose: automated tests of R1–R8, for example a 403 for another employer's applicant, no application after closing and no duplicate. Benefit: shows the rules hold as shared code changes. Learning: plain test functions and Flask's test client.
- **Flask-Mail (optional, F21).** Purpose: email an applicant when a status changes and a recruiter when an application arrives. Benefit: applicants need not log in to learn their progress. Integration: SMTP settings and one send call from M4, using a local test mail server in demonstrations.